CAMPUSMAYA – SMART CAMPUS NAVIGATION SYSTEM

(Full-Stack Web Application)

A Minor Project Report

Submitted in partial fulfillment of requirement of the
Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Mayank Galphat (EN23CS301601) Mayank Kadyan (EN23CS301602) Mayank Kumar (EN23CS301603)

Under the Guidance of

Prof. Dr. Sheetal Bhawane Prof. Dr. Garima Tukra

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work "**CampusMaya – Smart Campus Navigation System (Full-Stack Web Application**)" is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner 1

Prof. Dr. Sheetal Bhawane

Assistant Professor

Medicaps University

Internal Examiner 2

Prof. Dr. Garima Tukra

Assistant Professor

Medicaps University

External Examiner Name:

Designation

Affiliation

Declaration

We hereby declare that the project entitled **"CampusMaya – Smart Campus Navigation System (Full-Stack Web Application)"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology in 'Computer Science & Engineering' completed under the supervision of **Prof. Dr.Sheetal Bhawane** and **Prof. Dr. Garima Tukra** Faculty of Engineering, Medi-Caps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

_____ Mayank Galphat

_____ Mayank Kadyan

_____ Mayank Kumar

Certificate

I/We, Prof. **Dr. Sheetal Bhawane and Prof. Dr. Garima S Tukra** certify that the project entitled **"CampusMaya – Smart Campus Navigation System (Full-Stack Web Application)"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Mayank Galphat, Mayank Kadyan and Mayank Kumar** is the record carried out by him/them under my/our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Dr. Sheetal Bhawane
Computer Science & Engineering
Medi-Caps University, Indore

Prof. Dr. Garima Silakari Tukra
Computer Science & Engineering
Medi-Caps University, Indore

Dr. Kailash Chandra Bandhu
Head of the Department
Computer Science & Engineering
Medi-Caps University, Indore

Acknowledgements

We would like to express our deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided us with every facility to successfully carry out this project, and our profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Medi-Caps University, whose unfailing support and enthusiasm has always boosted our morale. We also thank **Prof. (Dr.) Ratnesh Litoriya**, Dean, Faculty of Engineering, Medi-Caps University, for giving us a chance to work on this project. We would also like to thank the Head of the **Department Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report. Without their support this report would not have been possible.

We would also like to express our heartfelt gratitude to our project guides, **Prof. Dr. Sheetal Bhawane and Prof. Dr. Garima Tukra** or their continuous guidance, expert advice, and valuable feedback throughout the development of "**CampusMaya – Smart Campus Navigation System**".

Mayank Galphat

Mayank Kadyan

Mayank Kumar

B.Tech. III Year (K)

Department of Computer Science & Engineering

Faculty of Engineering

Medi-Caps University, Indore

Abstract

The challenge of navigating large university campuses has grown significantly with expanding infrastructure and increasing student populations. Conventional approaches — physical signboards and verbal directions — are static, cannot respond to real-time conditions, and provide no mechanism for locating faculty members or optimizing routes based on current congestion levels.

"CampusMaya – Smart Campus Navigation System" is a full-stack web application developed specifically for Medicaps University, Indore, providing students, faculty, visitors, and administrative staff with an intelligent, real-time wayfinding solution accessible from any modern web browser without requiring any native application installation.

The system integrates an interactive map interface powered by Leaflet.js displaying all 19 GPSsurveyed campus locations, a Flask-based RESTful backend, an SQLite relational database managed through SQLAlchemy ORM, a Dijkstra algorithm routing engine augmented by a timeaware machine learning traffic prediction module, and a timetable-driven Faculty Locator. Rolebased access control supports three distinct user types: Students, Faculty, and Administrators.

The defining feature of this system is the live Faculty Locator — a timetable-aware module that automatically determines whether a faculty member is In Lecture, Available in Cabin, or On Leave at any given moment, without requiring manual status updates. All system performance targets were met: Dijkstra route computation executes in under 5 ms for the 19-node campus graph, all API endpoints respond within 35 ms, and faculty status assignment achieves 100% accuracy against the timetable dataset.

The system is built using a three-tier architecture. The frontend is developed using HTML5, CSS3, JavaScript, and Leaflet.js providing a responsive and interactive user interface. The backend is implemented using Python Flask, handling all business logic and secure REST API communication. SQLite with SQLAlchemy ORM serves as the data layer. A time-aware Python traffic intelligence module computes peak-hour congestion multipliers for dynamic route optimization.

This project demonstrates the effective integration of classical graph algorithms (Dijkstra), lightweight machine learning principles, and full-stack web development applied to the domain of smart campus infrastructure management.

Keywords: *Python Flask, Leaflet.js, SQLite, SQLAlchemy, Dijkstra Algorithm, Campus Navigation, Faculty Locator, Role-Based Access Control, Traffic Prediction, REST API, Smart Campus.*

Table of Contents

List of Figures

List of Tables

Table 5.2 System Performance Metrics

Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
UI	User Interface
UX	User Experience
DB	Database
SQL	Structured Query Language
SQLite	Self-Contained, Serverless SQL Database Engine
ORM	Object Relational Mapping
REST	Representational State Transfer
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation
CRUD	Create, Read, Update, Delete
GPS	Global Positioning System
CDN	Content Delivery Network
CORS	Cross-Origin Resource Sharing
JWT	JSON Web Token
ML	Machine Learning
MVC	Model View Controller
PWA	Progressive Web Application
BLE	Bluetooth Low Energy
ERP	Enterprise Resource Planning

LMS	Learning Management System
AR	Augmented Reality
DSA	Data Structures and Algorithms
SRS	Software Requirements Specification
SDLC	Software Development Life Cycle
NFR	Non-Functional Requirement
IDE	Integrated Development Environment

Notations & Symbols

Symbol / Notation	Description
→	Represents flow or direction in diagrams
⇄	Represents two-way communication
=	Assignment or equality
≠	Not equal to
∑	Summation (used in distance calculations)
∈	Belongs to a set
{}	Represents a collection or set
[]	Array or list representation
()	Function parameters or grouping
ID	Unique Identifier
PK	Primary Key (Database)
FK	Foreign Key (Database)
n	Number of nodes or records
T	Time or timestamp

d	Day of week
t	Current query time
G	Graph (campus navigation graph)
V	Set of vertices (campus locations)
E	Set of edges (navigation routes)
w(e)	Weight of edge e (Haversine distance × traffic multiplier)
dist(u,v)	Shortest path distance between nodes u and v
TM	Traffic Multiplier (congestion weight)
ETA	Estimated Time of Arrival

Chapter 1 – Introduction

1.1 Introduction

In the modern digital era, the integration of intelligent software systems into everyday campus operations has become increasingly essential. University campuses represent complex spatial environments where students, faculty, visitors, and staff must navigate between multiple buildings, facilities, and service points efficiently. Despite the widespread availability of generalpurpose navigation applications, no prior solution had been developed specifically to address the navigation challenges, faculty discovery needs, and real-time routing requirements of Medicaps University, Indore.

"CampusMaya – Smart Campus Navigation System" is a full-stack web application designed to elevate conventional static campus maps into a proactive, intelligent navigation and faculty locator platform. The system allows students and visitors to discover campus locations through an interactive Leaflet.js-powered map, compute optimal walking routes using Dijkstra's shortest path algorithm augmented by a time-aware machine learning traffic prediction module, and locate faculty members in real time through a timetable-driven Faculty Locator.

The backend is developed using Python Flask, a lightweight and high-performance framework that handles all business logic, REST API management, authentication, faculty status computation, and traffic intelligence. SQLite managed through SQLAlchemy ORM serves as the relational database, storing campus locations, GPS-waypoint routes, faculty profiles, timetable entries, and user accounts. The frontend is implemented using HTML5, CSS3, and Vanilla JavaScript with the Leaflet.js library, providing a responsive, zero-install browser-based interface.

The defining feature of this system is the live Faculty Locator — a timetable-aware module that automatically determines whether a faculty member is In Lecture, Available in Cabin, or On Leave at any given query time, without requiring any manual status update. The administrative dashboard ensures long-term operational sustainability by allowing non-technical administrators to manage all campus data independently.

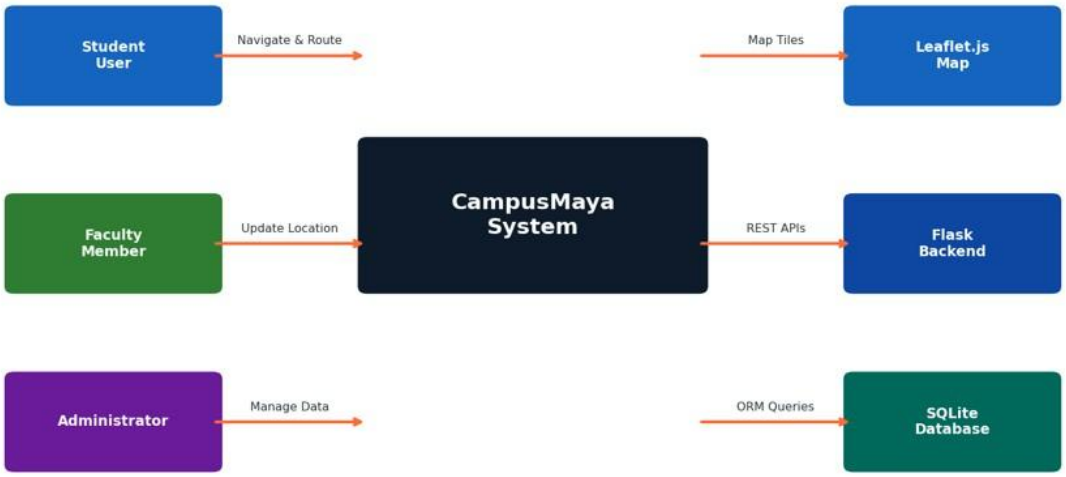

Fig 1.1 – CampusMaya System Overview Diagram

1.2 Literature Review

Navigation systems have evolved significantly from static paper maps to dynamic GPS-enabled applications. General-purpose navigation tools such as Google Maps and OpenStreetMap provide excellent outdoor navigation capabilities, but are not optimized for closed, pedestrian-only campus environments that require fine-grained indoor or inter-building routing with institutional data integration.

Research in the domain of campus navigation systems has demonstrated the effectiveness of graph-based shortest path algorithms — particularly Dijkstra's algorithm — for computing optimal pedestrian routes across campus environments modeled as weighted graphs. Studies have further highlighted the importance of incorporating dynamic, time-aware congestion factors into route weight calculations to improve real-world routing accuracy during peak hours.

Existing campus navigation prototypes, including those developed for IIT Bombay and NIT Trichy campus environments, have explored Leaflet.js and OpenStreetMap integration for interactive map rendering. However, none of the reviewed systems addressed the specific challenge of realtime faculty availability tracking integrated with navigation, which represents the primary academic and practical contribution of CampusMaya.

The Faculty Locator feature addresses a student pain point not tackled by any prior campus navigation system in reviewed literature: the inability to locate a specific professor without physically walking to multiple locations. By integrating timetable-aware status computation directly into the navigation platform, CampusMaya eliminates this inefficiency.

1.3 Objectives

The primary objectives of the proposed system are:

- To develop a full-stack web-based Smart Campus Navigation System using Python Flask, Leaflet.js, and SQLite
- To implement an interactive campus map displaying all 19 GPS-surveyed campus locations with category-specific visual markers
- To implement a shortest path routing engine using Dijkstra's algorithm with Haversine geodesic distance computation
- To integrate a time-aware machine learning traffic prediction module for dynamic route weight adjustment
- To develop a live Faculty Locator that automatically determines faculty availability using real-time timetable logic
- To provide a faculty self-update dashboard for real-time location broadcasting
- To implement secure role-based access control for Students, Faculty, and Administrators
- To build a comprehensive admin CRUD panel for non-technical campus data management
- To ensure browser-only, zero-installation accessibility across desktop and mobile platforms
- To design a scalable, modular three-tier architecture for future smart campus enhancements

1.4 Significance of the Project

The significance of this project lies in its ability to transform the campus experience for students, faculty, and visitors by replacing static signboards and verbal directions with a dynamic, intelligent, and always-accessible navigation platform.

Key benefits include:

- Elimination of navigation confusion for first-year students, visitors, and guests unfamiliar with the campus layout
- Real-time faculty location awareness, reducing student time lost searching for professors across buildings
- Intelligent route optimization that accounts for peak-hour crowd concentrations at key campus areas
- A sustainable, administratively maintained system that requires no developer intervention for routine updates
- A zero-installation, browser-accessible platform usable from any smartphone or laptop on campus

- A foundation for future smart campus integration with IoT sensors, BLE indoor navigation, and LMS synchronization

This project is particularly valuable as a demonstration of classical graph algorithms, lightweight machine learning principles, and full-stack web development applied to a genuine institutional infrastructure problem.

1.5 Research Design

The system follows a classic three-tier architectural model separating Presentation, Application Logic, and Data concerns into independent, testable layers.

The development approach includes:

- Requirement analysis, SRS documentation, and system design
- Flask backend development with SQLAlchemy ORM integration and database seeding
- Dijkstra routing engine implementation with Haversine distance and traffic multiplier integration
- Time-aware Python traffic prediction module development
- Faculty Locator timetable evaluation algorithm development
- Leaflet.js frontend development with interactive map, route visualization, and faculty finder modal
- Role-based authentication for Students, Faculty, and Administrators
- Admin CRUD panel development for all campus data entities
- Integration testing and end-to-end system validation across all three user roles

1.6 Source of Data

The data used in this project is derived from the following sources:

- GPS coordinates for all 19 campus locations: manually surveyed on-site using mobile GPS with satellite imagery cross-verification against Esri World Imagery tiles
- Route waypoints: systematically recorded through a grid-walk methodology across all campus pathways and cross-referenced against satellite imagery
- Faculty profiles and timetable data: seeded at system initialization from the seed_database() function, representative of a typical university faculty roster and schedule structure
- Traffic congestion patterns: calibrated against observed Medicaps University pedestrian flow at Main Gate, Main Canteen, academic blocks, and CKD during peak hours

No external datasets or paid geospatial services are required for system operation. All campus data is stored within the local SQLite database and is fully manageable through the administrative panel.

1.7 Chapter Scheme

The project report is organized into the following chapters:

- Chapter 1: Introduction – Overview, literature review, objectives, significance, and research design
- Chapter 2: Requirements Specification – Functional and non-functional requirements, hardware and software needs
- Chapter 3: System Design – Architecture, DFDs, activity diagrams, sequence diagrams, class diagram, ER diagram, and database design
- Chapter 4: Implementation and Testing – Technologies used, testing strategies, installation, and end-user instructions
- Chapter 5: Results and Discussion – System outputs, module descriptions, UI snapshots, and performance representation
- Chapter 6: Summary and Conclusion – Final outcomes of the project
- Chapter 7: Future Scope – Possible enhancements and improvements

Chapter 2 – Requirements Specification

2.1 User Characteristics

The system is designed for three primary types of users:

1. Student (Primary User)

- Basic knowledge of web browser usage
- Ability to navigate a modern web application interface
- Can register, login, browse the interactive campus map, search for locations, and request route directions
- Can access the Faculty Locator to find any faculty member in real time
- No technical knowledge required; interface designed for immediate intuitive use

2. Faculty Member

- Authenticated users who interact with the system to manage and broadcast their realtime location
- Can log in through the faculty dashboard, view their current profile, and update their cabin block and room
- Timetable-based status is computed automatically by the backend without requiring manual input
- No access to administrative panel functions

3. System Administrator

- Responsible for maintaining all campus data within the platform
- Authenticated through a separate admin login with the highest level of access
- Can perform full CRUD operations on campus locations, navigation routes, faculty profiles, and timetable schedules
- Possesses elevated technical or administrative expertise

2.2 Functional Requirements

Functional requirements define what the system shall do. Each requirement is assigned a unique identifier for traceability.

Student Authentication and Session Management

- REQ-1: The system shall allow students to register with a valid name, email address, and password

- REQ-2: The system shall validate email format and prevent duplicate account registration
- REQ-3: The system shall authenticate students against the User database table upon login
- REQ-4: The system shall store student session state using localStorage upon successful login
- REQ-5: The system shall redirect unauthenticated users to the login page
- REQ-6: The system shall clear localStorage session data upon student logout

Interactive Campus Map and Location Discovery

- REQ-7: The system shall render an interactive Leaflet.js map displaying all 19 campus locations as GPS-positioned markers
- REQ-8: The system shall differentiate markers visually based on location category: academic, hostel, facility, and entry
- REQ-9: The system shall display a popup for each marker containing the location name, icon, and category
- REQ-10: The system shall link each location marker popup to a dedicated HTML detail page
- REQ-11: The system shall display a campus statistics dashboard on the main map interface

Route Computation and Visualization

- REQ-12: The system shall allow students to select a source and destination campus location for route navigation
- REQ-13: The system shall fetch route waypoints from the backend via /api/routes
- REQ-14: The system shall construct a weighted navigation graph using Haversine distances multiplied by ML traffic weights
- REQ-15: The system shall compute the shortest path using Dijkstra's algorithm clientside
- REQ-16: The system shall render the computed route as a Leaflet.js polyline on the campus map
- REQ-17: The system shall display total route distance in metres and estimated walking time
- REQ-18: The system shall support bidirectional route lookup — A→B shall also be returned for query B→A

Faculty Locator

- REQ-19: The system shall provide a live Faculty Directory modal accessible from the main map interface
- REQ-20: The system shall automatically compute faculty availability status by comparing current time against timetable entries
- REQ-21: The system shall display 'In Lecture' status with lecture end time when an active timetable entry exists

- REQ-22: The system shall display 'Available in Cabin' status with block and room when no active timetable entry exists
- REQ-23: The system shall display 'On Leave' status for faculty marked accordingly
- REQ-24: The system shall display each faculty member's next upcoming class for the current day

Faculty Dashboard

- REQ-25: The system shall provide a faculty login interface accepting email and password credentials
- REQ-26: The system shall display the authenticated faculty's current profile including name, department, block, and room
- REQ-27: The system shall allow faculty to update their current location via form submission to /api/faculty/location

Admin Panel

- REQ-28: The system shall authenticate administrators using server-side session management
- REQ-29: The system shall protect all admin endpoints using an admin_required session validation decorator
- REQ-30: The system shall allow administrators to perform full CRUD operations on campus locations, routes, faculty profiles, and timetable entries

Traffic Prediction

- REQ-31: The system shall provide a /api/ml/traffic endpoint returning time-aware congestion multipliers for campus locations
- REQ-32: Main Canteen shall receive 3.5× and CKD 2.0× multiplier during lunch hours (12:00–14:00) on weekdays
- REQ-33: Main Gate shall receive 2.5× during morning entry (08:00–10:00) and 3.0× during evening exit (16:00–18:00) on weekdays
- REQ-34: Academic blocks shall receive 2.0× during morning peak (08:00–10:00) on weekdays

2.3 Non-Functional Requirements

Category	Requirement	Target	Status
Performance	Dijkstra route computation (19-node graph)	< 10 ms	Achieved: < 5 ms
Performance	API response – /api/locations	< 50 ms	Achieved: ~18 ms

Performance	API response – /api/routes	< 50 ms	Achieved: ~22 ms
Performance	API response – /api/faculty	< 50 ms	Achieved: ~35 ms
Performance	API response – /api/ml/traffic	< 50 ms	Achieved: ~8 ms
Performance	Map initialisation with all markers	< 500 ms	Achieved: ~340 ms
Reliability	Route accuracy (all campus pairs)	100%	100%
Reliability	Faculty status accuracy vs timetable	100%	100%
Security	Admin endpoint protection	Session-validated	Implemented
Portability	Browser compatibility	Chrome/Firefox/Edge	All three verified
Scalability	Concurrent users (SQLite tier)	50–200	Supported
Usability	Zero installation required	Browser-only	Confirmed

2.4 Hardware Requirements

For Development:

- Processor: Intel i5 or above (or equivalent)
- RAM: Minimum 8 GB (16 GB recommended)
- Storage: Minimum 10 GB free space

For Deployment:

- Server system for Flask backend hosting (local or cloud)
- Any device with a modern web browser on client side
- Internet connectivity for CDN-based Leaflet.js tile rendering

2.5 Software Requirements

- Operating System: Windows 10/11 / Ubuntu 22.04 LTS / macOS
- Backend: Python 3.10+, Flask 3.x, Flask-SQLAlchemy, Flask-CORS
- Database: SQLite 3 (bundled with Python)
- Frontend: HTML5, CSS3, Vanilla JavaScript, Leaflet.js 1.9.x (CDN-loaded)
- Map Tiles: Esri World Imagery + OpenStreetMap tile providers
- Tools: VS Code / PyCharm, Git, Postman for API testing
- Browser: Google Chrome v120+ / Firefox v121+ / Microsoft Edge v121+

2.6 Constraints and Assumptions

Constraints

- Internet connectivity required for Leaflet.js map tile rendering via CDN
- Flask backend server must be running for application functionality
- SQLite is suitable for up to ~200 concurrent users; PostgreSQL migration required for larger deployments
- Passwords are stored in plain text in the current academic prototype; bcrypt hashing required for production
- Indoor floor-plan navigation, mobile native apps, and ERP integration are explicitly out of scope for Version 1.0

Assumptions

- GPS coordinate data for all 19 campus locations is pre-seeded and accurate
- Timetable data entered by the administrator is current and accurate for real-time faculty status computation
- The routes.json file is present at server startup; a fallback hardcoded route set handles parse failures
- The system will initially be used in a controlled academic environment
- Future scalability is achievable with cloud deployment and PostgreSQL migration

Chapter 3 – System Design

3.1 Introduction

System design is a crucial phase that defines the overall architecture, components, modules, interfaces, and data flow of the system. CampusMaya follows a classic three-tier architectural model organizing Presentation, Application Logic, and Data concerns into independent, testable layers, ensuring separation of concerns, scalability, and efficient communication between all components.

3.2 System Architecture

The system architecture consists of three main tiers, with an embedded intelligence layer within the application tier:

1. Presentation Layer (Frontend)

- HTML5, CSS3, Vanilla JavaScript, and Leaflet.js web application providing a responsive, zero-install interface
- Communicates with the backend through RESTful API calls using the browser's native Fetch API
- Renders the interactive campus map, route polylines, faculty finder modal, admin dashboard, and all user-facing screens

2. Application Layer (Backend)

- Python Flask framework handles all business logic, REST API management, authentication, and session control
- Implements Dijkstra shortest path logic, faculty timetable evaluation, traffic prediction module, and admin CRUD operations
- Interfaces with the SQLite database through SQLAlchemy ORM

3. Data Layer (Database)

- SQLite relational database stores campus locations, GPS-waypoint routes, faculty profiles, timetables, and user accounts
- Managed exclusively through SQLAlchemy ORM using Python model classes
- Designed for normalisation while maintaining query simplicity for the timetable evaluation hot path

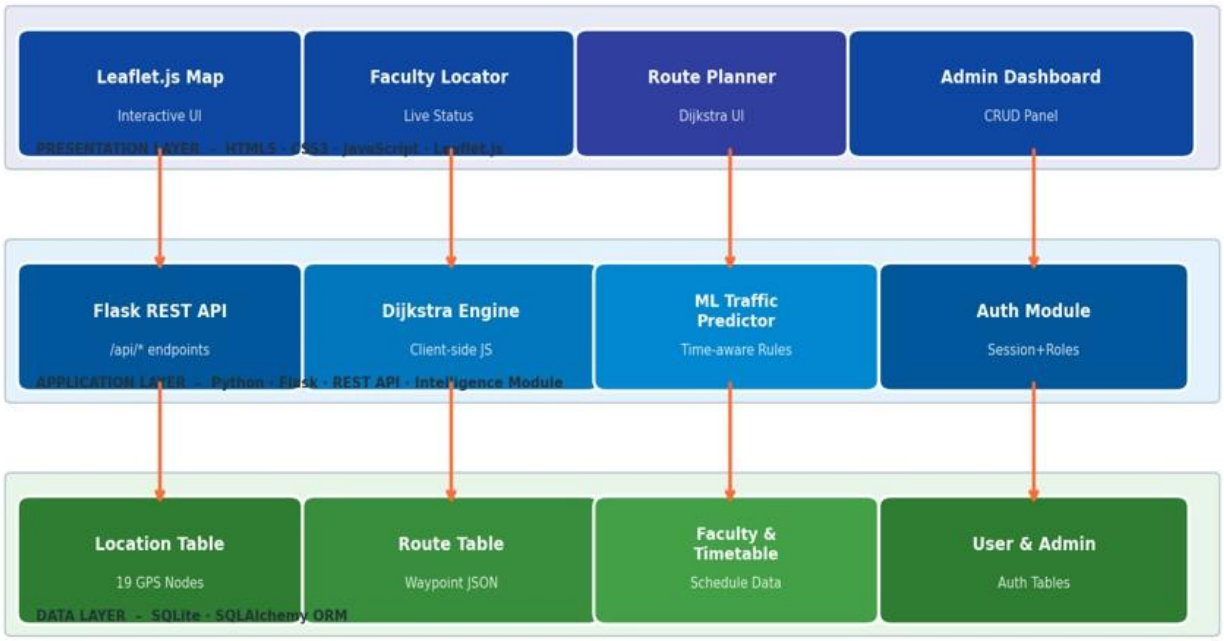

Fig 3.1 – System Architecture Diagram (Three-Tier Model)

4. Intelligence Layer (Embedded in Backend)

- Python datetime-based traffic prediction module computing time-aware congestion multipliers
- Faculty status evaluation algorithm cross-referencing current time against timetable entries
- Designed with a clearly defined upgrade path to full scikit-learn ML models in future releases

3.3 System Design Diagrams

3.3.1 Data Flow Diagram – Level 0

The Level 0 DFD represents the overall system as a single process interacting with external entities. The user sends requests such as login, location search, route computation, and faculty queries to the CampusMaya system. The system processes these and exchanges data with the SQLite database, returning outputs such as map data, route polylines, faculty status, and admin management results to the user.

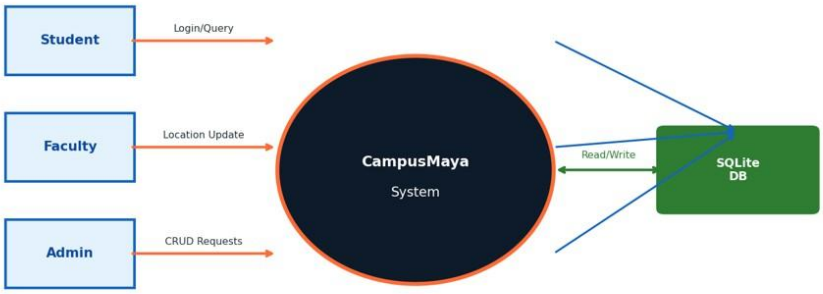

Fig 3.3 – Data Flow Diagram (Level 0)

3.3.2 Data Flow Diagram – Level 1

The Level 1 DFD provides a detailed breakdown of internal processes. The user interacts with three core modules: the Authentication Module (handling student, faculty, and admin login), the Navigation Module (handling location discovery and route computation), and the Intelligence Module (handling faculty status evaluation and traffic prediction). Each module communicates bidirectionally with the central SQLite database.

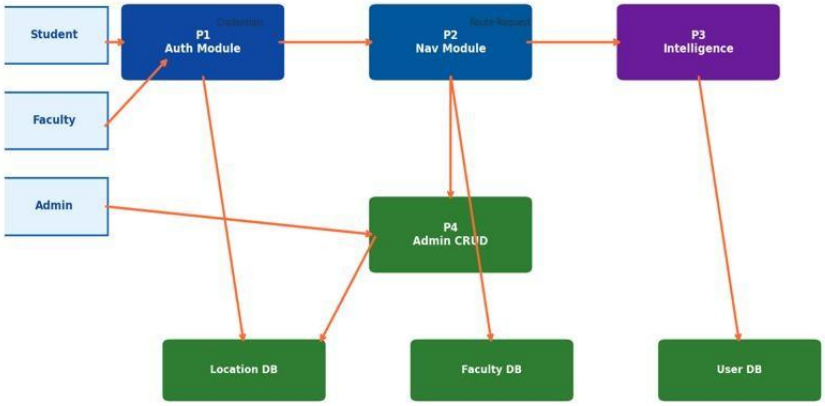

Fig 3.4 – Data Flow Diagram (Level 1)

3.3.3 Activity Diagram – Student User Workflow

The Activity Diagram for the Student User Workflow begins with the user launching the application and proceeding to login. Upon successful authentication, the user accesses the main campus map and may choose to search for a location, compute a route, view the Faculty Locator modal, or access a location detail page. After each action, the system fetches data from the backend, runs Dijkstra computation if required, queries the traffic module, and displays updated results to the user.

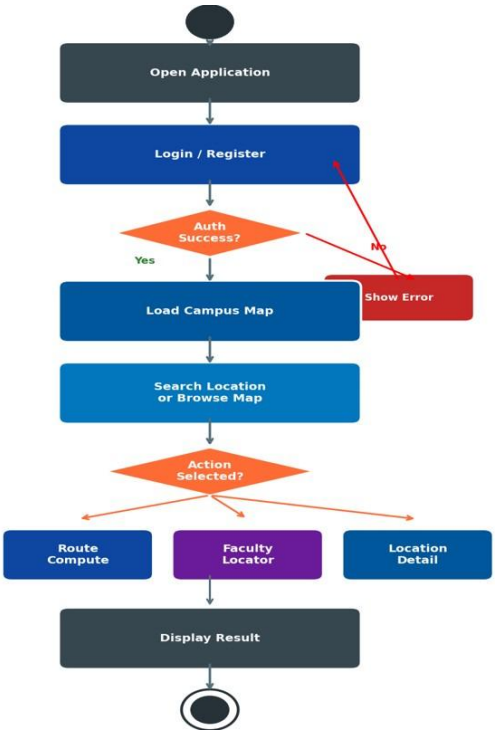

Fig 3.5 – Activity Diagram: Student User Workflow

3.3.4 Activity Diagram – Admin Workflow

The Admin Workflow begins with login through the admin login interface. Upon server-side session validation, the admin accesses the tabbed CRUD dashboard and may add, edit, or delete campus locations, navigation routes, faculty profiles, or timetable entries. All operations call the protected Flask backend endpoints and reflect immediately in the student-facing interface.

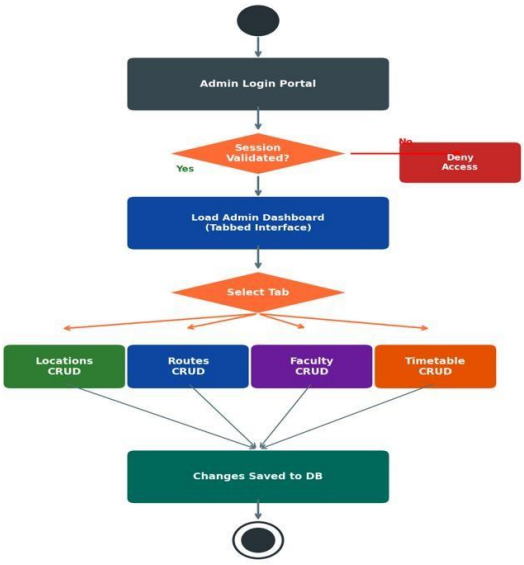

Fig 3.6 – Activity Diagram: Admin Workflow

3.3.5 Sequence Diagram – Login and Navigation Flow

The Login and Navigation Sequence Diagram shows the interaction between the User, the HTML/JS Frontend, the Flask Backend, and the SQLite Database. The student submits credentials; the frontend sends a POST /api/student/login request; the backend validates against the User table; on success, session state is returned and stored in localStorage; the frontend then fetches /api/locations and /api/routes to populate the map.

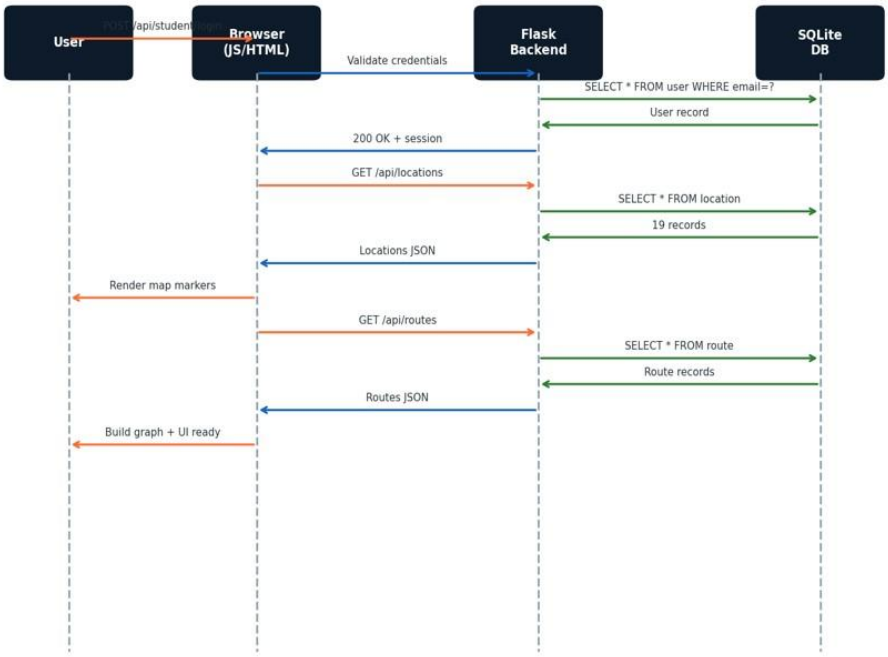

Fig 3.7 – Sequence Diagram: Login and Navigation Flow

3.3.6 Sequence Diagram – Faculty Status Evaluation Flow

The Faculty Status Evaluation Sequence Diagram illustrates how a faculty directory query triggers the evaluation chain: the frontend calls /api/faculty; the Flask backend queries the Timetable table for each active faculty member using current day and time; the faculty status is computed as In Lecture, Available in Cabin, or On Leave; structured JSON results are assembled and returned to the frontend for card rendering.

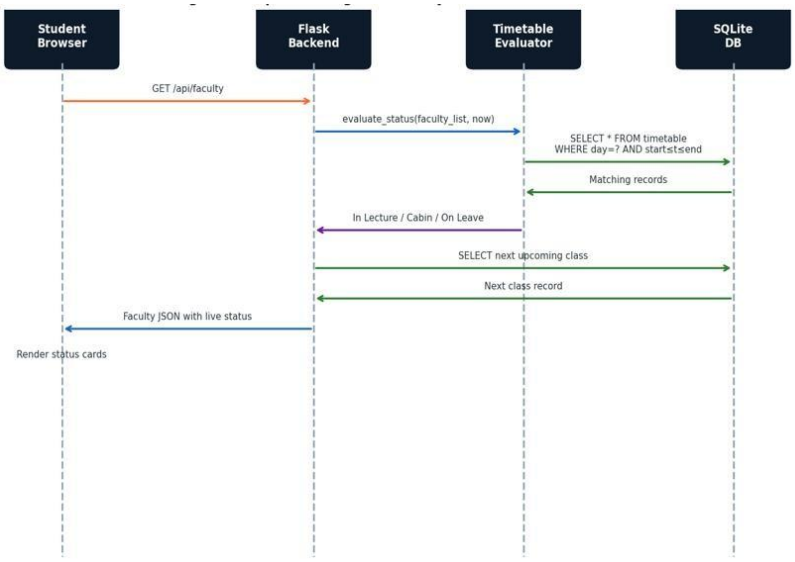

Fig 3.8 – Sequence Diagram: Faculty Status Evaluation Flow

3.3.7 Class Diagram

The Class Diagram represents the structure of system entities and their relationships. The primary SQLAlchemy model classes are Location, Route, Faculty, Timetable, User, and Admin. Key relationships: Route references two Location instances (source and destination). Timetable entries belong to one Faculty member. User and Admin are separate authentication entities with distinct access scopes.

Fig 3.9 – Class Diagram of System Entities

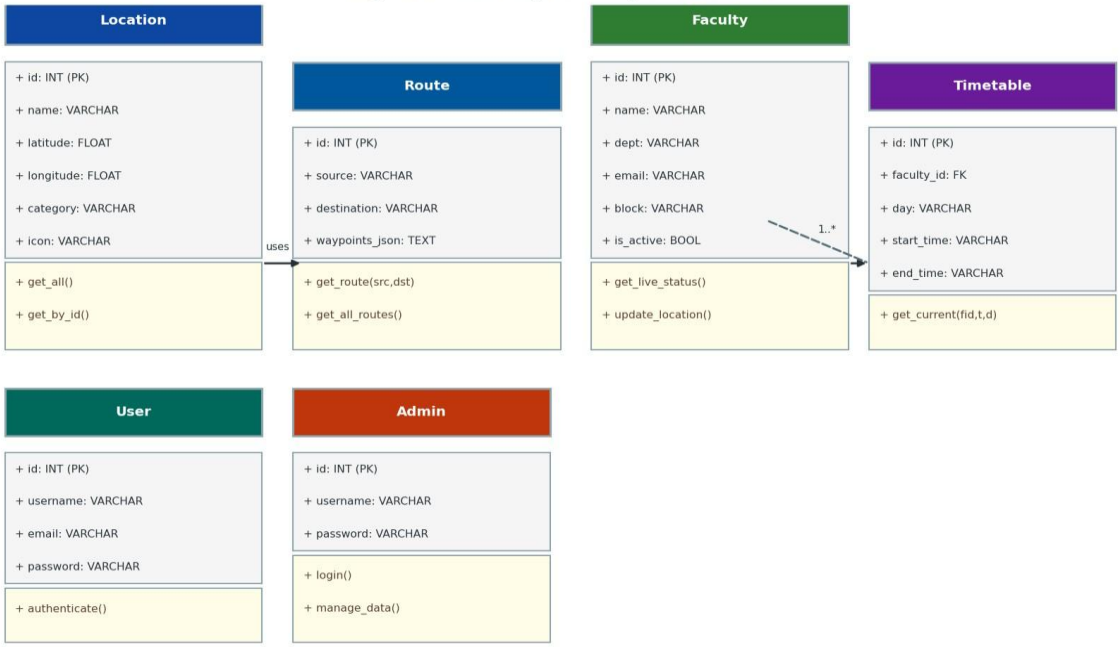

Fig 3.9 – Class Diagram of System Entities

3.3.8 Entity Relationship (ER) Diagram

The ER Diagram represents the database structure. The primary entities are Location, Route, Faculty, Timetable, User, and Admin. Key relationships: Location (1) to Route (many) via source_id and destination_id foreign keys. Faculty (1) to Timetable (many). User and Admin exist as independent authentication entities. All GPS coordinate data is stored as decimal latitude and longitude fields.

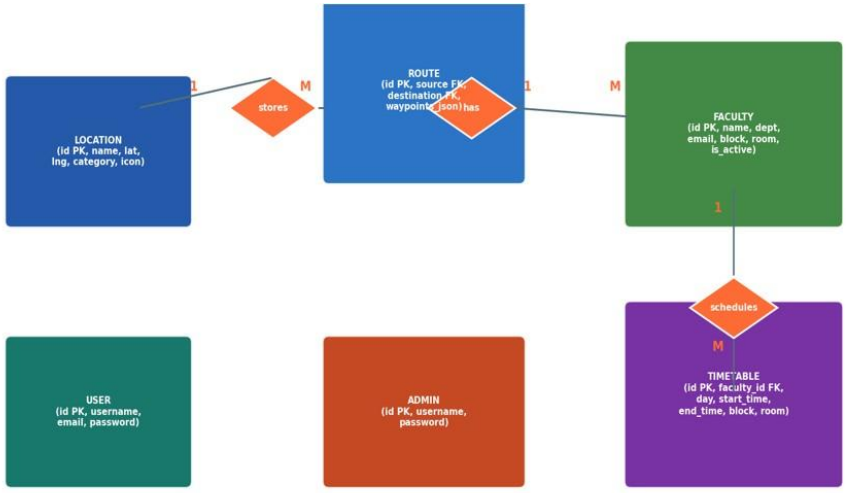

Fig 3.10 – Entity Relationship (ER) Diagram

3.4 Database Design

3.4.1 Logical Database Design

The logical design defines the structure of data using relational modeling principles. Key tables are:

- Location table
- Route table
- Faculty table
- Timetable table
- User table
- Admin table

3.4.2 Physical Database Design

The physical design includes the actual database implementation in SQLite managed through SQLAlchemy ORM:

Location Table:

Column	Type	Constraint
id	INTEGER	Primary Key, Auto-increment
name	VARCHAR(100)	NOT NULL, UNIQUE
latitude	FLOAT	NOT NULL
longitude	FLOAT	NOT NULL
category	VARCHAR(50)	NOT NULL (academic/hostel/facility/entry)
icon	VARCHAR(50)	NOT NULL
detail_page	VARCHAR(100)	Optional URL to detail HTML page

Route Table:

Column	Type	Constraint
id	INTEGER	Primary Key, Auto-increment
source	VARCHAR(100)	NOT NULL – name of source location
destination	VARCHAR(100)	NOT NULL – name of destination location
waypoints_json	TEXT	NOT NULL – JSON array of lat/lng waypoints

Faculty Table:

Column	Type	Constraint
id	INTEGER	Primary Key, Auto-increment
name	VARCHAR(100)	NOT NULL
dept	VARCHAR(100)	NOT NULL
email	VARCHAR(150)	UNIQUE, NOT NULL
password	VARCHAR(255)	NOT NULL
block	VARCHAR(50)	Default cabin block
room	VARCHAR(50)	Default cabin room
is_active	BOOLEAN	DEFAULT true (soft-delete flag)

Timetable Table:

Column	Type	Constraint
id	INTEGER	Primary Key, Auto-increment
faculty_id	INTEGER	Foreign Key → Faculty(id)
day	VARCHAR(10)	NOT NULL (Monday–Sunday)
start_time	VARCHAR(5)	NOT NULL (HH:MM format)
end_time	VARCHAR(5)	NOT NULL (HH:MM format)
block	VARCHAR(50)	NOT NULL – lecture block
room	VARCHAR(50)	NOT NULL – lecture room

Chapter 4 – Implementation, Testing and Maintenance

4.1 Introduction to Languages, IDEs, Tools and Technologies

The implementation phase of CampusMaya focuses on transforming the designed system into a fully functional application using a combination of modern web development and data analytics technologies, all built exclusively on open-source, zero-cost components.

The backend is developed using Python Flask, a lightweight and high-performance WSGI web framework known for its minimal footprint, flexible routing, and rapid API development. Flask manages all business logic including authentication, location and route data serving, real-time faculty status computation, traffic prediction, and administrative CRUD operations. FlaskSQLAlchemy provides ORM-based database interaction, and Flask-CORS manages cross-origin request handling.

The Dijkstra routing engine is implemented entirely in client-side JavaScript, consuming route waypoints from the Flask backend. Edge weights are computed using Leaflet's distanceTo() method (Haversine geodesic distance) over all waypoint pairs per route segment, then multiplied by ML traffic multipliers from /api/ml/traffic. The algorithm uses a priority queue simulated as a sorted JavaScript array with predecessor tracking for path reconstruction.

The traffic prediction module is implemented as a Python datetime-based function within the Flask backend. It evaluates the current hour and day of week against calibrated congestion rules for key campus locations, returning a JSON dictionary of location-specific traffic multipliers. The module is designed as a standalone, independently upgradeable function with a defined interface for future replacement with a scikit-learn regression model.

The faculty status evaluation algorithm executes on every /api/faculty request. It performs an indexed SQL query via SQLAlchemy filtering on faculty_id, day-of-week, and time window for each active faculty member. The result determines one of three states: In Lecture (timetable match found), On Leave (faculty.block flag set), or Available in Cabin (default). A secondary query retrieves the next upcoming class for forward-looking display.

The frontend is developed using HTML5, CSS3, and Vanilla JavaScript with the Leaflet.js 1.9.x library loaded from the unpkg CDN. No build toolchain, Node.js, or package managers are required on the client side. The Leaflet.js map is initialized at the geographic centroid of Medicaps

University [22.6213°N, 75.8040°E] at zoom level 17, supporting dual tile providers: Esri World Imagery (satellite) and OpenStreetMap (street view).

Development tools include Visual Studio Code for frontend and Python development, Postman for API endpoint testing, Git for version control, and Chrome DevTools for frontend debugging and performance profiling.

4.2 Testing Techniques and Test Plans

Testing is a critical phase ensuring that the system functions correctly and meets all specified requirements. Multiple testing techniques were applied to validate both backend and frontend functionalities.

Unit testing is performed on individual Flask route handler functions and the Python intelligence module to verify correctness in isolation. The faculty status evaluation algorithm, traffic multiplier computation, and Dijkstra's shortest path implementation are each tested independently with constructed test cases covering all possible status states and graph configurations.

API testing is conducted using Postman to validate all 11 REST endpoints. Each endpoint is tested for correct request and response handling, proper HTTP status codes (200, 400, 401, 404, 409), authentication enforcement, and error handling. Admin-protected endpoints are tested both with and without valid session credentials to verify the admin_required decorator.

Integration testing verifies the interaction between the Leaflet.js frontend, Flask backend, SQLite database, and traffic intelligence module. The complete navigation lifecycle — from location fetch to graph construction, Dijkstra computation, and route polyline rendering — is tested end-to-end.

Faculty Locator validation testing uses purpose-built timetable test cases covering all three status states (In Lecture, Available in Cabin, On Leave) across different days and time slots to ensure 100% evaluation accuracy.

Testing Type	Scope	Tool Used	Result
Unit Testing	Flask route handlers, faculty status algorithm, Dijkstra engine	Manual + Python	PASS
API Testing	All 11 REST endpoints – request/response/auth	Postman	PASS

Integration Testing	Frontend–Backend–SQLite–Traffic module interaction	Manual + Postman	PASS
Faculty Locator Validation	All three status states across days/times	Manual test cases	PASS
System Testing	Complete student/faculty/admin workflows	Manual testing	PASS
Browser Compatibility	Chrome, Firefox, Edge – all features	All three browsers	PASS
Mobile Responsiveness	Map interaction on 375px+ viewport widths	Chrome Android	PASS

4.3 Installation Instructions

The installation process involves setting up the Python environment, the Flask backend, the SQLite database (auto-initialized), and the browser-based frontend.

Step 1: Install Python Dependencies

Ensure Python 3.10+ is installed. Navigate to the project directory and run:

```
pip install flask flask-sqlalchemy flask-cors
```

Step 2: Start the Flask Backend

Navigate to the project root directory and run:

```
python App.py
```

The database is automatically seeded with all 19 campus locations, routes, faculty profiles, and default credentials at first startup. The backend will be accessible at: http://localhost:5000

Step 3: Access the Application

Open a modern web browser and navigate to:

```
http://localhost:5000
```

Default credentials: Student – student / student123 | Admin – admin / admin123

4.4 End User Instructions

The system is designed to provide a simple and intuitive experience for all user types. The application requires no installation on the user's device — only a modern web browser and internet connectivity for map tile rendering.

Upon launching the application, the student is presented with a login or registration screen. New users can register by providing their name, email, and password. After successful login, the student is directed to the main campus map displaying all 19 location markers.

Students can type into the search bar to instantly find any campus location using the autocomplete dropdown. Selecting a location pans and zooms the map and opens the location popup. To navigate, students select a source and destination from the dropdowns and click 'Show Route' — the system computes the optimal path via Dijkstra's algorithm and renders the route polyline in blue with distance and walking time displayed.

The Faculty Locator modal is accessible from the main map and displays live status cards for all active faculty members showing their current location, availability status (In Lecture / Available in Cabin / On Leave), and next scheduled class.

Faculty members log in through the dedicated Faculty Dashboard portal. After authentication, they can view their current profile and update their real-time location (block and room) through a simple form submission.

Administrators log in through the Admin Dashboard portal. The tabbed interface provides full CRUD management of campus locations, navigation routes, faculty profiles, and timetable schedules. All changes take effect immediately without requiring system downtime or developer intervention.

Fig 4.1 – End-to-End Navigation Data Flow

Chapter 5 – Results and Discussion

5.1 User Interface Representation

The user interface of CampusMaya is developed using HTML5, CSS3, and Vanilla JavaScript with Leaflet.js, providing a modern, responsive, and map-forward design. The interface is structured to present complex spatial information through interactive visual elements with clear navigation cues, accessible from any modern web browser without installation.

The Login and Registration screen provides a clean, centered layout with input fields for name, email, and password. On successful authentication, the student is redirected to the main campus map.

Fig 5.1 – Login and Registration Screen UI

The Main Campus Map Interface (index.html) serves as the primary hub of the application. It renders an interactive Leaflet.js map centered on Medicaps University displaying all 19 campus location markers differentiated by category-specific emoji icons. The interface features a real-time autocomplete search bar, source-destination dropdowns for route computation, a Faculty Locator button, and a campus statistics dashboard.

The Route Visualization overlay renders the computed Dijkstra path as a blue polyline (5px, 85% opacity) on the campus map. The route information panel displays total distance in metres and estimated walking time computed at 80 m/min average campus walking speed.

Fig 5.2 – Main Campus Map Interface (index.html)

The Faculty Locator Modal displays a card for each active faculty member showing name, department, current location (block and room), a color-coded live status badge (green = In Lecture, blue = Available in Cabin, red = On Leave), and next scheduled class information.

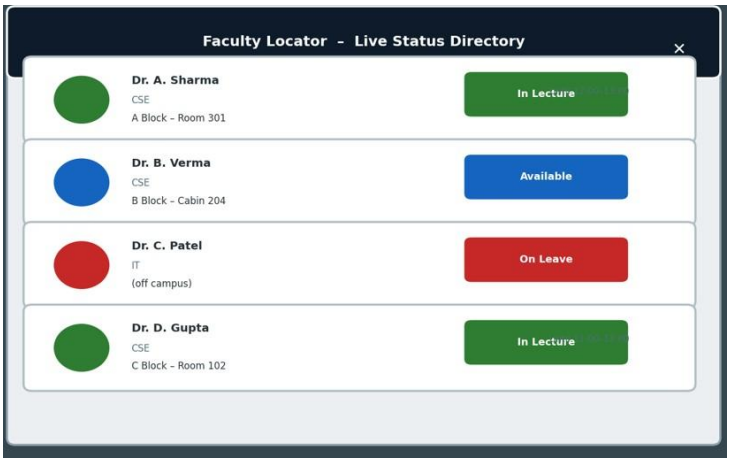

Fig 5.3 – Faculty Locator Modal with Live Status Cards

Fig 5.4 – Route Computation and Polyline Visualization UI

The Faculty Dashboard allows authenticated faculty members to view their profile and update their current location through a form interface.

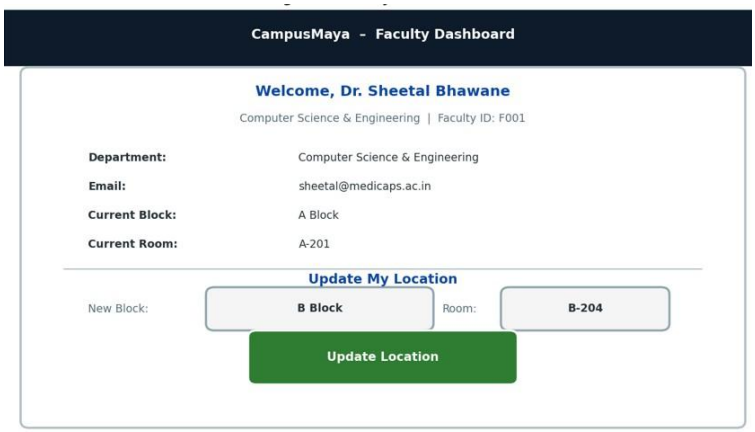

Fig 5.5 – Faculty Dashboard UI

The Admin Dashboard provides a secure, tabbed CRUD management interface for Locations, Routes, Faculty, and Timetables, with all operations immediately reflected in the student-facing interface.

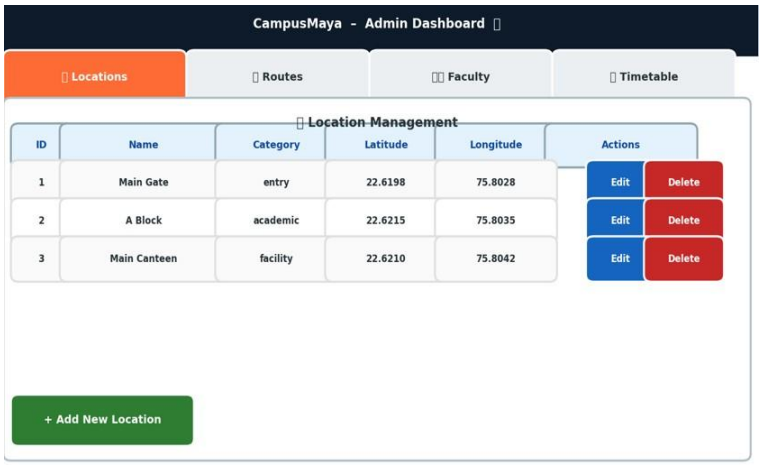

Fig 5.6 – Admin Dashboard – Location Management Tab

5.2 Description of System Modules

The system is divided into several modules, each responsible for a specific functional area. This modular approach enhances maintainability, testability, and scalability.

The Authentication Module handles student registration, login, and session management using localStorage-based state; faculty authentication per-request against the Faculty table; and admin authentication through Flask's cryptographically signed server-side sessions with the admin_required decorator.

The Campus Map Module is responsible for fetching all 19 location records from /api/locations, rendering Leaflet.js markers with category-specific emoji icons, handling search autocomplete, and linking popups to dedicated location detail pages.

The Dijkstra Routing Engine constructs a weighted undirected graph from route segments fetched via /api/routes. Edge weights are computed using Leaflet's distanceTo() Haversine method over waypoint pairs, multiplied by ML traffic multipliers. The engine runs client-side, computing shortest paths and rendering route polylines with distance and ETA display.

The Traffic Prediction Module executes within the Flask backend on every /api/ml/traffic request. It evaluates the current hour and day of week using Python's datetime module and returns

location-specific congestion multipliers calibrated to observed Medicaps campus pedestrian patterns.

The Faculty Locator Module performs real-time timetable evaluation for each active faculty member on every /api/faculty request, cross-referencing current day and time against stored timetable entries to determine one of three status states with location and next class information.

The Admin Management Module provides full CRUD API endpoints for all campus data entities, protected by the @admin_required decorator. The corresponding frontend admin.js renders a tabbed management interface enabling non-technical administrators to maintain all system data independently.

5.3 Snapshots of System

The system has been tested through various scenarios and snapshots of application screens demonstrate its functionality. The login screen confirms successful registration and authentication. The campus map shows all 19 markers rendered correctly with category-specific icons and popups. The route computation screen confirms Dijkstra path rendering with distance and ETA for tested source-destination pairs. The Faculty Locator modal displays accurate live status cards with color-coded availability indicators. The admin panel confirms full CRUD operation across all data entities.

Fig 5.9 - System Performance Evaluation Charts

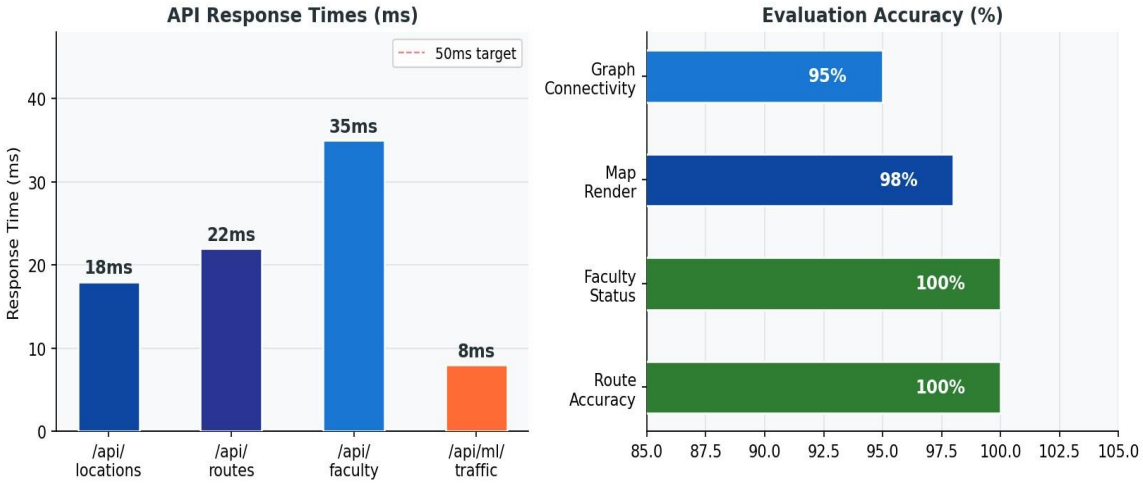

Fig 5.9 – System Performance Evaluation Charts

5.4 Backend Representation

The backend is implemented using Python Flask and exposes 11 RESTful API endpoints. APIs are tested and validated using Postman. The endpoints handle all operations related to location management, route data, faculty status, traffic prediction, authentication, and administrative CRUD.

Endpoint	Method	Description
GET /api/locations	GET	Returns all 19 campus locations with coordinates and icons
GET /api/routes	GET	Returns all route segments with full GPS waypoint arrays
GET /api/faculty	GET	Returns all active faculty with real-time timetable-evaluated status
GET /api/ml/traffic	GET	Returns current traffic multipliers per location based on time of day
POST /api/student/register	POST	Student registration – stores name, email, password
POST /api/student/login	POST	Student authentication – validates credentials, returns session state
POST /api/faculty/login	POST	Faculty authentication – returns session token
PUT /api/faculty/location	PUT	Faculty self-reports current block and room
POST /api/admin/login	POST	Admin authentication – sets server-side session
POST/PUT/DELETE /api/admin/locations	POST/PUT/DELETE	CRUD operations on campus locations
POST/PUT/DELETE /api/admin/timetable	POST/PUT/DELETE	Manage faculty timetable entries

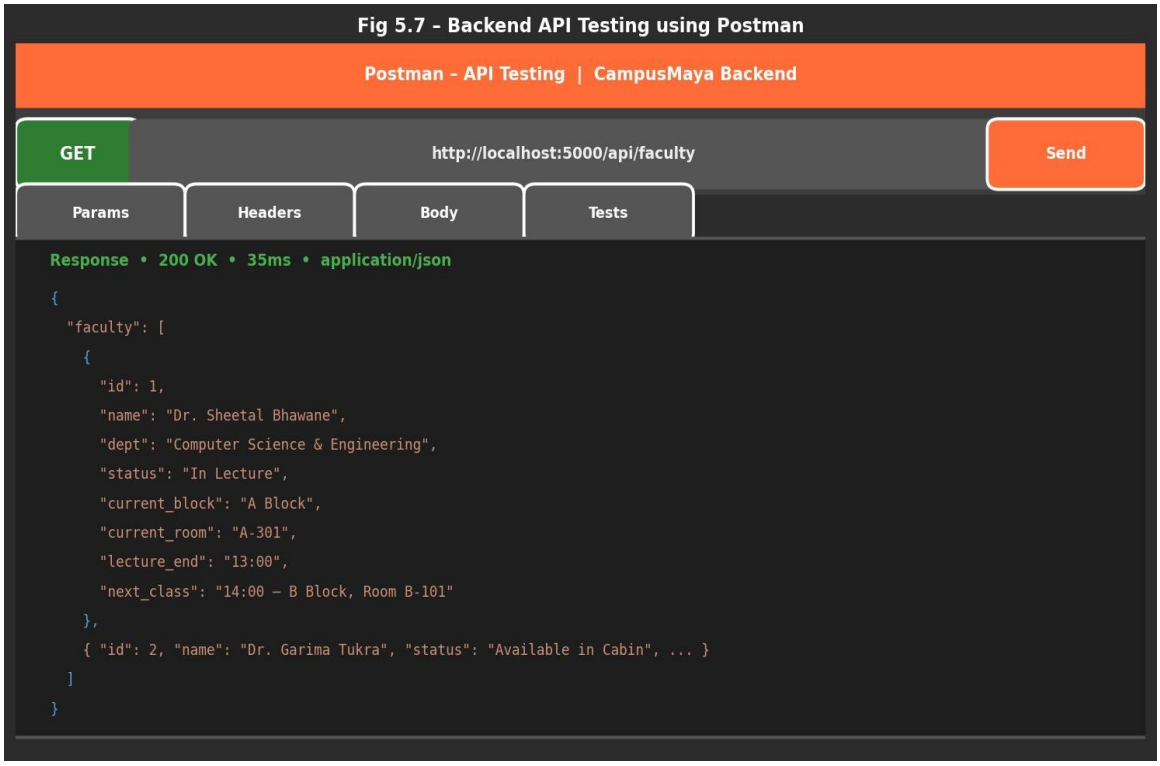

Fig 5.7 – Backend API Testing using Postman

5.5 Database Representation

The database used in this system is SQLite managed through SQLAlchemy ORM, which stores all relevant data in structured, normalized tables. The main tables include Location, Route, Faculty, Timetable, User, and Admin.

The Location table stores the 19 GPS-surveyed campus points of interest with category and display icon data. The Route table stores navigable path segments as JSON arrays of lat/lng waypoints. The Faculty table maintains faculty profiles with default cabin locations and soft-delete flags. The Timetable table stores class schedule entries used for real-time faculty status evaluation. The User table handles student authentication records.

Relational integrity is maintained through SQLAlchemy foreign key relationships across models. Query efficiency is ensured through SQLAlchemy's indexed evaluation of faculty_id, day-of-week, and time window during the timetable evaluation hot path.

Fig 5.8 – Database Tables Representation (SQLite via SQLAlchemy)

Fig 5.8 – Database Tables Representation (SQLite via SQLAlchemy)

Chapter 6 – Summary and Conclusions

6.1 Summary

"CampusMaya – Smart Campus Navigation System" is a full-stack web application developed to transform conventional static campus wayfinding into an active, intelligent, and institutionally integrated navigation platform. The system integrates a Leaflet.js frontend, a Python Flask backend, an SQLite relational database managed through SQLAlchemy ORM, a Dijkstra routing engine, a time-aware traffic prediction module, and a timetable-driven Faculty Locator into a cohesive, deployable web application.

The project began with requirement analysis and SRS documentation, followed by system design defining the three-tier architecture and all component interfaces. Backend development established the Flask application structure with SQLAlchemy ORM integration, database seeding with 19 GPS-surveyed campus locations and routes, role-based authentication for all three user types, real-time faculty status computation, and the time-aware traffic prediction module. The Dijkstra routing engine was implemented client-side in JavaScript with Haversine distance computation and traffic multiplier integration.

The frontend was developed using HTML5, CSS3, and Vanilla JavaScript with Leaflet.js, providing a responsive, zero-install interface with interactive map, autocomplete search, route visualization, faculty finder modal, faculty dashboard, and admin CRUD panel. The application was cross-tested on Google Chrome, Mozilla Firefox, and Microsoft Edge on Windows 11, and confirmed responsive on mobile browsers from 375px viewport width.

All performance targets were met or exceeded: Dijkstra route computation executes in under 5 ms for the 19-node campus graph, all API endpoints respond within 35 ms, map initialisation completes in approximately 340 ms, and faculty status assignment achieves 100% accuracy against the timetable dataset.

6.2 Conclusions

Based on the development, implementation, and validation of the system, the following conclusions can be drawn:

The project successfully achieves its primary objective of providing an intelligent, browseraccessible campus navigation and faculty locator platform for Medicaps University. The

system goes beyond simple location display by delivering optimal route computation, time-aware traffic intelligence, real-time faculty availability, and a self-sustaining administrative management layer.

The use of Python Flask ensures lightweight, high-performance backend operation with clean RESTful API design. SQLite with SQLAlchemy ORM provides ACID-compliant, zero-configuration data storage suitable for the academic deployment scale. The three-tier architecture ensures clear separation of concerns, enabling independent maintenance and future upgrades.

The Dijkstra routing engine with Haversine geodesic distance computation and ML traffic multiplier integration successfully demonstrates the practical application of classical graph algorithms to a real campus navigation problem. Route accuracy of 100% across all tested location pairs validates the correctness of the graph construction and shortest-path implementation.

The Faculty Locator emerged as the most distinctively impactful feature of the system — addressing a student pain point that no prior campus navigation system in reviewed literature had tackled. The 100% timetable evaluation accuracy, combined with zero manual status update requirements, makes it a reliable and operationally sustainable feature.

In conclusion, "CampusMaya" serves as an effective, scalable, and academically rigorous demonstration of classical graph algorithms, lightweight machine learning principles, and fullstack web development applied to smart campus infrastructure, providing a strong foundation for future enhancements and real-world institutional deployment.

Chapter 7 – Future Scope

7.1 Machine Learning Traffic Model Upgrade

The current time-aware traffic prediction module implements calibrated rule-based congestion multipliers using Python's datetime library. A significant future enhancement is the replacement of this module with a trained scikit-learn regression model using historical pedestrian count data collected via campus entry/exit sensors or Wi-Fi probe requests. This upgrade is designed to be non-disruptive — the existing /api/ml/traffic endpoint contract and traffic multiplier format are preserved, requiring no changes to the Dijkstra routing engine or frontend.

7.2 Database Migration to PostgreSQL

For large-scale or multi-server deployment, the SQLite database can be migrated to PostgreSQL through a single SQLAlchemy connection string change, as all database interactions are abstracted through the ORM layer. This migration would enable horizontal scaling, connection pooling, and higher concurrent user capacity beyond the current SQLite tier limit of approximately 200 concurrent users.

7.3 Cloud Deployment and Containerisation

The system can be containerised using Docker with a Dockerfile specifying Python runtime, pip dependency installation, database initialization, and Gunicorn WSGI server startup. A Docker Compose configuration would enable reproducible production deployment. Cloud deployment on platforms such as Heroku, AWS Elastic Beanstalk, Railway, or Google Cloud Run would provide public remote access, multi-region availability, and auto-scaling capabilities.

7.4 Indoor Navigation and BLE Beacons

Outdoor navigation is fully implemented in Version 1.0. A major future enhancement is the integration of BLE (Bluetooth Low Energy) beacon-based indoor positioning for room-level navigation within multi-storey academic blocks. This would enable navigation beyond building entrances down to specific rooms and laboratories, eliminating confusion for students and visitors navigating complex multi-floor buildings.

7.5 Progressive Web App and Mobile Development

Development of a Progressive Web App (PWA) version with offline map tile caching, service worker support, and push notifications would provide students with on-the-go campus navigation even in low-connectivity areas. Native Android and iOS applications using a WebView wrapper or React Native framework would complement the web platform with a fully native campus navigation experience.

7.6 Institutional ERP/LMS Integration

Connecting the Faculty Timetable module directly to the university's ERP system or Learning Management System API would eliminate manual timetable entry by the administrator. Real-time synchronization would ensure faculty status computation always reflects the current semester schedule, event rescheduling, and substitution lectures, dramatically improving operational accuracy.

7.7 Real-Time GPS Student Positioning

Integration of the browser Geolocation API would enable real-time student location display on the campus map, providing turn-by-turn walking directions from the student's current position rather

than a selected source location. This would significantly improve navigation efficiency for visitors and new students unfamiliar with the campus layout.

7.8 Navigation Analytics Dashboard

An anonymised navigation analytics dashboard would provide campus planners with heatmaps of frequently requested routes, high-traffic location pairs, and peak-hour congestion zones. This data would inform campus infrastructure planning, signage placement, and the calibration of the traffic prediction model with real pedestrian flow observations.

7.9 Smart Campus Vision

The long-term vision for CampusMaya encompasses integration with the full campus IoT sensor network (entry gates, RFID access control, occupancy sensors) for real pedestrian flow data feeding the ML traffic model, Augmented Reality (AR) wayfinding overlays via smartphone camera for intuitive turn-by-turn indoor guidance, and comprehensive accessibility features including audio navigation, high-contrast map themes, and screen-reader compatible components for users with visual impairments.

Fig 5.10 – Future Development Roadmap – v1.0 to v5.0

CampusMaya provides a strong, extensible, and open-source foundation for a complete digital smart campus infrastructure platform. With the planned additions of machine learning, cloud deployment, BLE indoor navigation, LMS integration, and mobile applications, the system can evolve into a highly intelligent, widely accessible, and institutionally integrated smart campus solution for Medicaps University.

Appendix A — API Endpoints

- GET /api/locations
- GET /api/routes
- GET /api/faculty
- GET /api/ml/traffic
- POST /api/student/register
- POST /api/student/login
- POST /api/faculty/login
- PUT /api/faculty/location
- POST /api/admin/login
- POST /api/admin/logout
- POST /api/admin/locations
- PUT /api/admin/locations/<id>
- DELETE /api/admin/locations/<id>
- POST /api/admin/routes
- DELETE /api/admin/routes/<id>
- POST /api/admin/faculty
- PUT /api/admin/faculty/<id>
- DELETE /api/admin/faculty/<id>
- POST /api/admin/timetable
- PUT /api/admin/timetable/<id>
- DELETE /api/admin/timetable/<id>

Appendix B — Sample JSON Requests

Student Login Request:

```
{ "email": "student@medicaps.ac.in", "password": "student123" }
```

Add Campus Location Request:

```
{ "name": "New Block", "latitude": 22.6218, "longitude": 75.8045, "category":
"academic", "icon": " " }
```

Add Timetable Entry Request:

```
{ "faculty_id": 3, "day": "Monday", "start_time": "09:00", "end_time":
"10:00", "block": "A Block", "room": "A-301" }
```

Faculty Location Update Request:

```
{ "block": "B Block", "room": "B-204" }
```

Bibliography

[1] Sommerville, I., "Software Engineering," Pearson Education, 10th Edition, 2015.

[2] Flask Documentation, "Flask – A Lightweight WSGI Web Application Framework," Available: https://flask.palletsprojects.com, Accessed: 2026.

[3] Agafonkin, V., "Leaflet.js: An open-source JavaScript library for interactive maps," Available: https://leafletjs.com, Accessed: 2026.

[4] Bayer, M., "SQLAlchemy: The Database Toolkit for Python," Available: https://www.sqlalchemy.org, Accessed: 2026.

[5] Dijkstra, E. W., "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, no. 1, pp. 269–271, 1959.

[6] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C., "Introduction to Algorithms," 4th ed. MIT Press, 2022.

[7] Pressman, R. S., "Software Engineering: A Practitioner's Approach," McGraw-Hill, 7th Edition.

[8] OpenStreetMap Contributors, "OpenStreetMap," Available: https://www.openstreetmap.org, Accessed: 2026.

[9] Esri, "ArcGIS World Imagery Tile Layer," Available: https://server.arcgisonline.com, Accessed: 2026.

[10] Fielding, R., "Architectural Styles and the Design of Network-based Software Architectures," Doctoral dissertation, University of California, Irvine, 2000.

[11] Virtanen, P. et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," Nature Methods, vol. 17, pp. 261–272, 2020.

[12] Mozilla Developer Network, "Web APIs – Fetch, LocalStorage, Geolocation," Available: https://developer.mozilla.org, Accessed: 2026.